MEVN Redis GraphQL Socket.IO

Nice — I'll give you a full, ready-to-adopt **TypeScript workflow** for the stack you requested: **MEVN + Redis + Socket.IO + GraphQL** with an `src/index.ts` that supports **HTTPS using PEM certs when provided** and **falls back to HTTP in dev**. Included:

- Folder structure
- `docker-compose.yml` (Node server, Redis, MongoDB, optional cert volume)
- TypeScript server code (Redis client, GraphQL schema/resolvers, Socket.IO wiring)
- Frontend snippets (Socket client, Pinia store, Apollo client)
- `tsconfig.json`, `package.json`, Dockerfiles, .env example
- Notes on running locally and in Docker

I kept it pragmatic and idiomatic so you can copy/paste, adapt, and deploy.

Project layout

```pgsql
project-root/
├─ docker-compose.yml
├─ server/
│  ├─ Dockerfile
│  ├─ package.json
│  ├─ tsconfig.json
│  ├─ src/
│  │  ├─ index.ts
│  │  ├─ redisClient.ts
│  │  ├─ graphql/
│  │  │  ├─ schema.ts
│  │  │  └─ resolvers.ts
│  │  └─ types/
│  │     └─ index.d.ts
│  └─ .env.example
├─ client/
│  ├─ Dockerfile
│  ├─ package.json
│  ├─ src/
│  │  ├─ socket.ts
│  │  ├─ apolloClient.ts
│  │  └─ stores/sessionStore.ts
└─ README.md
```

Top-level `docker-compose.yml`

```yaml
version: "3.8"
services:
  server:
    build: ./server
    env_file: ./server/.env.example
    ports:
      - "4000:4000"
      - "443:443"    # optional if you mount PEMs and want HTTPS on 443
    depends_on:
      - redis
      - mongo
    volumes:
      - ./server/certs:/etc/ssl/certs:ro    # optional: put server.pem/key.pem
here for HTTPS

  redis:
    image: redis:7
    command: ["redis-server", "--save", "60", "1"]
    ports:
      - "6379:6379"
    volumes:
      - redis-data:/data

  mongo:
    image: mongo:6
    restart: unless-stopped
    ports:
      - "27017:27017"
    volumes:
      - mongo-data:/data/db

  client:
    build: ./client
    ports:
      - "5173:5173"
    depends_on:
      - server

volumes:
  redis-data:
  mongo-data:
```

Server (TypeScript)

server/package.json

```
json
```

```
{
  "name": "realtime-server",
  "version": "1.0.0",
  "type": "module",
  "scripts": {
    "dev": "ts-node-dev --respawn --transpile-only src/index.ts",
    "build": "tsc",
    "start": "node dist/index.js"
  },
  "dependencies": {
    "apollo-server-express": "^3.11.1",
    "cors": "^2.8.5",
    "dotenv": "^16.0.0",
    "express": "^4.18.2",
    "graphql": "^16.6.0",
    "http": "^0.0.1-security",
    "redis": "^4.6.5",
    "socket.io": "^4.7.1",
    "socket.io-redis": "^6.1.0",
    "mongodb": "^5.8.0"
  },
  "devDependencies": {
    "@types/express": "^4.17.19",
    "@types/node": "^20.3.1",
    "ts-node-dev": "^2.0.0",
    "typescript": "^5.1.3"
  }
}
```

> Note: `socket.io-redis` is included as an optional adapter if you later want to scale across Node instances (for Socket.IO <-> Redis adapter). With Socket.IO v4 you can use `@socket.io/redis-adapter` in production — swap if needed.

server/tsconfig.json

```
json
```

```
{
  "compilerOptions": {
    "target": "ES2020",
    "module": "ES2020",
    "moduleResolution": "node",
    "outDir": "dist",
    "esModuleInterop": true,
    "forceConsistentCasingInFileNames": true,
    "strict": true,
    "skipLibCheck": true,
    "resolveJsonModule": true,
    "sourceMap": true
  },
  "include": ["src/**/*"]
}
```

server/.env.example

```
ini
```

```
PORT=4000
HOST=0.0.0.0

# Optional HTTPS certs (if provided, server will use HTTPS)
SSL_KEY_PATH=/etc/ssl/certs/key.pem
SSL_CERT_PATH=/etc/ssl/certs/cert.pem

REDIS_URL=redis://redis:6379
MONGO_URI=mongodb://mongo:27017/mydb
ALLOWED_ORIGIN=http://localhost:5173
```

server/src/redisClient.ts

```ts
// simple redis client module
import { createClient } from "redis";
import dotenv from "dotenv";
dotenv.config();

const url = process.env.REDIS_URL ?? "redis://localhost:6379";

export const redisClient = createClient({ url });
redisClient.on("error", (err) => console.error("Redis Client Error", err));

export const createRedisSubscriber = async () => {
  const sub = redisClient.duplicate();
  await sub.connect();
  return sub;
};

export const connectRedis = async () => {
  if (!redisClient.isOpen) {
    await redisClient.connect();
  }
};

export default redisClient;
```

server/src/graphql/schema.ts

```ts
import { gql } from "apollo-server-express";

export const typeDefs = gql`
  type Participant {
    id: ID!
    name: String
  }

  type Response {
    userId: ID!
    quizId: ID!
    answer: String
    timestamp: String
  }

  type Query {
    getResponses(quizId: ID!): [Response]
  }

  type Mutation {
    saveResponse(userId: ID!, quizId: ID!, answer: String!): Response
  }
`;
```

server/src/graphql/resolvers.ts

```ts
import { redisClient } from "../redisClient.js";

export const resolvers = {
  Query: {
    async getResponses(_: any, { quizId }: { quizId: string }) {
      const raw = await redisClient.hGetAll(`quiz:${quizId}`);
      return Object.values(raw || {}).map((r) => JSON.parse(r));
    }
  },
  Mutation: {
    async saveResponse(_: any, { userId, quizId, answer }: { userId: string;
quizId: string; answer: string }) {
      const response = { userId, quizId, answer, timestamp: new
Date().toISOString() };
      await redisClient.hSet(`quiz:${quizId}`, userId, JSON.stringify(response));
      // publish analytics update
      await redisClient.publish("analytics:update", JSON.stringify({ sessionId:
quizId, ...response }));
      return response;
    }
  }
};
```

`server/src/index.ts` — main server with HTTPS fallback

```ts
import express from "express";
import http from "http";
import https from "https";
import fs from "fs";
import cors from "cors";
import { Server as SocketIOServer } from "socket.io";
import dotenv from "dotenv";
import { ApolloServer } from "apollo-server-express";
import bodyParser from "body-parser";

import redisClient, { connectRedis, createRedisSubscriber } from
"./redisClient.js";
import { typeDefs } from "./graphql/schema.js";
import { resolvers } from "./graphql/resolvers.js";

dotenv.config();

const PORT = parseInt(process.env.PORT || "4000", 10);
const HOST = process.env.HOST || "0.0.0.0";
const ALLOWED_ORIGIN = process.env.ALLOWED_ORIGIN || "*";

async function start() {
  await connectRedis().catch((err) => {
    console.error("Failed to connect to Redis", err);
    process.exit(1);
  });

  // Express app
  const app = express();
  app.use(cors({ origin: ALLOWED_ORIGIN }));
  app.use(bodyParser.json());

  // Apollo server setup (apollo-server-express v3)
  const apollo = new ApolloServer({ typeDefs, resolvers });
  await apollo.start();
  app.use("/graphql", apollo.getMiddleware({ path: "/graphql", cors: false }));

  // Decide whether to start HTTPS server or HTTP fallback
  let server: http.Server | https.Server;
  const keyPath = process.env.SSL_KEY_PATH;
  const certPath = process.env.SSL_CERT_PATH;

  if (keyPath && certPath && fs.existsSync(keyPath) && fs.existsSync(certPath)) {
    const key = fs.readFileSync(keyPath);
    const cert = fs.readFileSync(certPath);
```

```
    server = https://createServer({ key, cert }, app);
    console.log("✅ Starting HTTPS server (certs provided)");
  } else {
    server = http.createServer(app);
    console.log("⚠️ HTTPS certs not found — starting HTTP server (dev
fallback)");
  }

  // Socket.IO attached to server
  const io = new SocketIOServer(server, {
    cors: { origin: ALLOWED_ORIGIN, methods: ["GET", "POST"] }
  });

  // Create subscriber for analytics pub/sub
  const subscriber = await createRedisSubscriber();
  await subscriber.subscribe("analytics:update", (message) => {
    try {
      const payload = JSON.parse(message);
      io.emit("analytics:update", payload);
    } catch (err) {
      console.error("Invalid analytics message", err);
    }
  });

  io.on("connection", (socket) => {
    console.log("Client connected", socket.id);

    socket.on("join-session", (sessionId: string) => {
      socket.join(sessionId);
      console.log(`Socket ${socket.id} joined ${sessionId}`);
    });

    socket.on("participant:submit-response", async (data) => {
      // store in redis and publish (also resolvers handle similar for GraphQL)
      try {
        const { sessionId, userId, answer } = data;
        const record = { userId, quizId: sessionId, answer, timestamp: new
Date().toISOString() };
        await redisClient.hSet(`quiz:${sessionId}`, userId,
JSON.stringify(record));
        await redisClient.publish("analytics:update", JSON.stringify({ sessionId,
...record }));
      } catch (err) {
        console.error("Error saving response", err);
      }
    });

    socket.on("facilitator:control", (data) => {
      // send to all clients in session
      const { sessionId, command } = data;
      io.to(sessionId).emit("control-command", command);
    });

    socket.on("disconnect", () => {
      console.log("Client disconnected", socket.id);
    });
```

```
  });

  server.listen(PORT, HOST, () => {
    console.log(`Server listening on ${HOST}:${PORT}`);
  });
}

start().catch((err) => {
  console.error("Server failed to start", err);
  process.exit(1);
});
```

Notes on server code:

- If you want to scale Socket.IO horizontally, replace the simple Redis pub/sub with `@socket.io/redis-adapter` and configure `io.adapter(createAdapter(pubClient, subClient))`.
- The GraphQL server is mounted at `/graphql`.
- The server will look for environment variables `SSL_KEY_PATH` and `SSL_CERT_PATH` and use HTTPS if both exist; otherwise it runs HTTP (dev fallback).
- For production, prefer proper TLS termination (load balancer / proxy) or provide valid certs.

server/Dockerfile

```dockerfile
FROM node:20-alpine

WORKDIR /app

COPY package*.json tsconfig.json ./
RUN npm ci

COPY . .

RUN npm run build

ENV NODE_ENV=production

EXPOSE 4000 443

CMD ["node", "dist/index.js"]
```

If you want dev image, use `ts-node-dev` and mount source code.

Frontend (Vue 3, TypeScript, Pinia, Apollo, Socket.IO)

You probably already have your Vue app; these are TypeScript-ready snippets.

`client/package.json` (essential deps)

```
json
```

```
{
  "name": "realtime-client",
  "version": "1.0.0",
  "scripts": {
    "dev": "vite",
    "build": "vite build",
    "preview": "vite preview"
  },
  "dependencies": {
    "vue": "^3.4.0",
    "pinia": "^2.0.30",
    "socket.io-client": "^4.7.1",
    "@apollo/client": "^3.8.0",
    "graphql": "^16.6.0",
    "vuetify": "^3.3.0"
  },
  "devDependencies": {
    "typescript": "^5.1.3",
    "vite": "^5.0.0"
  }
}
```

client/src/socket.ts (TypeScript)

```ts
import { io } from "socket.io-client";
import { reactive } from "vue";

const SERVER = import.meta.env.VITE_SERVER_URL ?? "http://localhost:4000";

export const socket = io(SERVER, { autoConnect: true });

export const socketState = reactive({
  connected: socket.connected
});

socket.on("connect", () => (socketState.connected = true));
socket.on("disconnect", () => (socketState.connected = false));
```

client/src/apolloClient.ts

```ts
import { ApolloClient, InMemoryCache, HttpLink } from "@apollo/client/core";

const GRAPHQL_URL = import.meta.env.VITE_GRAPHQL_URL ??
"http://localhost:4000/graphql";

export const apolloClient = new ApolloClient({
  link: new HttpLink({ uri: GRAPHQL_URL }),
  cache: new InMemoryCache()
});
```

`client/src/stores/sessionStore.ts` (Pinia)

```ts
import { defineStore } from "pinia";
import { socket } from "@/socket";
import { ref } from "vue";

export const useSessionStore = defineStore("session", () => {
  const analytics = ref<Record<string, any>>({});
  const responses = ref<any[]>([]);
  const sessionId = ref<string | null>(null);

  function init(id: string) {
    sessionId.value = id;
    socket.emit("join-session", id);
    socket.on("analytics:update", (data) => {
      if (data.sessionId === id) {
        analytics.value = data;
      }
    });
    socket.on("control-command", (cmd) => {
      // handle controls (start, stop, lock, etc)
      console.log("control", cmd);
    });
  }

  function submitResponse(userId: string, answer: string) {
    if (!sessionId.value) throw new Error("Session not initialized");
    const payload = { sessionId: sessionId.value, userId, answer };
    socket.emit("participant:submit-response", payload);
  }

  function facilitatorCommand(command: string) {
    if (!sessionId.value) throw new Error("Session not initialized");
    socket.emit("facilitator:control", { sessionId: sessionId.value, command });
  }

  return { analytics, responses, sessionId, init, submitResponse,
facilitatorCommand };
});
```

Example GraphQL usage (client side)

Use Apollo to call the `saveResponse` mutation or `getResponses` query when you want structured persistence / reporting:

```ts
import { gql } from "@apollo/client/core";
import { apolloClient } from "./apolloClient";

const SAVE_RESPONSE = gql`
  mutation SaveResponse($userId: ID!, $quizId: ID!, $answer: String!) {
    saveResponse(userId: $userId, quizId: $quizId, answer: $answer) {
      userId
      quizId
      answer
      timestamp
    }
  }
`;

await apolloClient.mutate({
  mutation: SAVE_RESPONSE,
  variables: { userId: "user123", quizId: "quiz001", answer: "B" }
});
```

> You can choose whether to write responses via Socket.IO only, GraphQL only, or both (double-write). In the server template above Socket.IO writes into Redis (hSet + publish), while GraphQL `Mutation.saveResponse` does the same. Choose one to avoid duplication, or make socket-only for realtime and GraphQL for historical queries.

TLS / Cert notes

- The server will check the two environment variables `SSL_KEY_PATH` and `SSL_CERT_PATH`. If both exist and files are readable, the server starts HTTPS using the provided PEM files.
- For local dev you can avoid certs entirely (HTTP). For testing HTTPS locally you can generate self-signed certs and mount them into `server/certs/`.
- In production, prefer TLS termination at LB or reverse proxy (NGINX/Traefik) and run Node behind it.

Quick commands

Locally (without Docker):

1. Server

```bash
cd server
cp .env.example .env
# set REDIS_URL to redis://localhost:6379
npm install
npm run dev
```

2. Client

```bash
cd client
npm install
npm run dev
```

With Docker:

```bash
docker-compose up --build
# optional: mount certs to server/certs for HTTPS at 443
```

Recommendations & next steps

- Add `@socket.io/redis-adapter` if you plan to horizontally scale Node app instances (will make Socket.IO rooms work across instances).
- Use Redis TTL, AOF/RDB configuration for persistence durability; or persist periodically from Redis to Mongo if you need long-term audit logs.
- Add authentication (JWT) and authorize socket connections (attach token during `io` connection).
- Consider a schema for analytics messages — right now we JSON-publish; you may want typed structures or a small event envelope (`type`, `payload`, `meta`).
- Add metrics/logging and graceful shutdown hooks for Redis/mongo/Socket.IO.

If you want, I can now:

- generate the **complete ready-to-run files** (I can paste each file fully here), or
- produce a **ready-to-run Docker Compose repo** layout with all files filled in (I'll paste the full content of each file), or
- **convert the Apollo usage to subscriptions** (GraphQL subscriptions) if you prefer structured subscriptions instead of Redis Pub/Sub → Socket.IO.